Java Datentypen

Autor

Luis Staudt

20. Dezember 2025

Inhaltsverzeichnis

Inhaltsverzeichnis 2
Java Datentypen 3
Primitive Datentypen 3
int - Ganzzahlen 4
double - Fließkommazahlen 6
boolean - Wahrheitswerte 8
char - Einzelnes Zeichen 10
byte - Sehr kleine Ganzzahlen 12
short - Kleine Ganzzahlen 13
long - Sehr große Ganzzahlen 14
float - Einfache Fließkommazahlen 15
String - Zeichenketten 16
Typkonvertierung 19
Übersicht: Primitive Datentypen 21
Arrays 21
array.length - Länge eines Arrays 23
array[index] - Element lesen 25
array[index] = wert - Element schreiben 27
Häufige Fehler 29

Java Datentypen

Primitive Datentypen

Java arbeitet mit verschiedenen Arten von Daten. Die einfachsten davon heißen pprimitive Datentypen“. Diese speichern direkte Werte wie Zahlen oder Ja/Nein-Antworten.

int - Ganzzahlen

Was ist int?

Der Datentyp `int` speichert ganze Zahlen - also Zahlen ohne Komma (wie 5, -10, oder 1000).

Wertebereich:

- Von: -2.147.483.648
- Bis: 2.147.483.647
- Speichergröße: 32 Bit (4 Bytes)

Syntax:

```
int variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
int alter = 25;
int temperatur = -5;
int punkte = 0;

// Ohne Initialisierung
int zahl;
zahl = 42;

// Berechnungen
int summe = 10 + 20;          // summe = 30
int differenz = 50 - 15;      // differenz = 35
int produkt = 6 * 7;          // produkt = 42
int quotient = 20 / 4;        // quotient = 5
int rest = 17 % 5;            // rest = 2

// Mit Variablen rechnen
int a = 10;
int b = 3;
int ergebnis = a + b;         // ergebnis = 13

// Inkrement und Dekrement
int zaehler = 5;
zaehler++;                    // zaehler = 6
zaehler--;                    // zaehler = 5

```

```
// Zusammengesetzte Operatoren
int wert = 10;
wert += 5;                    // wert = 15 (entspricht: wert = wert + 5)
wert -= 3;                    // wert = 12
wert *= 2;                    // wert = 24
wert /= 4;                    // wert = 6
```

Worauf achten:

- Wenn du eine int-Zahl durch eine int-Zahl teilst, bekommst du wieder eine int-Zahl: `5 / 2 = 2` (nicht 2.5! Die Nachkommastellen werden einfach abgeschnitten)
- Wenn die Zahl zu groß oder zu klein wird, springt sie auf die andere Seite um (das nennt man Überlauf)
- In einem Array ist der Anfangswert immer `0`
- Mit int kannst du keine Kommazahlen speichern

double - Fließkommazahlen

Was ist double?

Der Datentyp `double` speichert Kommazahlen - also Zahlen mit Nachkommastellen (wie 3.14, -5.5, oder 0.001).

Wertebereich:

- Sehr große und sehr kleine Zahlen mit ca. 15 Dezimalstellen Genauigkeit
- Von: $\pm 4.9 \times 10^{-324}$
- Bis: $\pm 1.7 \times 10^{308}$
- Speichergröße: 64 Bit (8 Bytes)

Syntax:

```
double variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
double pi = 3.14159;
double temperatur = -5.5;
double gewicht = 75.8;
double ergebnis = 0.0;

// Berechnungen
double summe = 10.5 + 20.3;        // summe = 30.8
double produkt = 2.5 * 4.0;        // produkt = 10.0
double quotient = 7.0 / 2.0;       // quotient = 3.5

// Wichtig: Division mit Nachkommastellen
double a = 5.0;
double b = 2.0;
double teil = a / b;               // teil = 2.5

// int zu double konvertieren
int ganzzahl = 5;
double kommazahl = ganzzahl / 2.0; // kommazahl = 2.5
// ODER
double kommazahl2 = (double) ganzzahl / 2; // kommazahl2 = 2.5

// Wissenschaftliche Notation
```

```
24 double klein = 1.5e-4;              // = 0.00015
25 double gross = 3.2e8;               // = 320000000.0
26
27 // Mathematische Funktionen
28 double wurzel = Math.sqrt(16.0);    // wurzel = 4.0
29 double potenz = Math.pow(2, 3);     // potenz = 8.0
30 double absolut = Math.abs(-5.5);    // absolut = 5.5
```

Worauf achten:

- In Java schreibst du einen Punkt statt Komma: `3.14` ist richtig, `3,14` ist falsch
- Computer können nicht immer exakt rechnen: manchmal kommt bei `0.1 + 0.2` nicht genau `0.3` heraus
- Wichtig: `5 / 2` ergibt `2`, aber `5.0 / 2.0` ergibt `2.5` - mindestens eine Zahl muss eine Kommazahl sein
- In einem Array ist der Anfangswert immer `0.0`
- Für Geld solltest du andere Datentypen verwenden (z.B. `BigDecimal`), weil Rundungsfehler problematisch sein können

boolean - Wahrheitswerte

Was ist boolean?

Der Datentyp `boolean` speichert Wahrheitswerte: entweder `true` (wahr) oder `false` (falsch).

Wertebereich:

- Nur zwei mögliche Werte: `true` oder `false`
- Speichergröße: 1 Bit (theoretisch), praktisch abhängig von JVM

Syntax:

```
boolean variablenName = true/false;
```

Beispiele:

```
// Deklaration und Initialisierung
boolean istWahr = true;
boolean istFalsch = false;
boolean hatBestanden = true;

// Vergleichsoperatoren ergeben boolean
int alter = 18;
boolean istErwachsen = (alter >= 18);      // true
boolean istKind = (alter < 12);            // false
boolean istGenau18 = (alter == 18);        // true
boolean istNicht18 = (alter != 18);        // false

// Logische Operatoren
boolean a = true;
boolean b = false;

boolean und = a && b;                      // false (beide muessen true
    sein)
boolean oder = a || b;                     // true (mindestens einer true)
boolean nicht = !a;                        // false (Negation)

// In Bedingungen
int punkte = 85;
boolean bestanden = punkte >= 50;
if (bestanden) {
    System.out.println("Bestanden!");
}
```

```java

// Kombinierte Bedingungen
int temperatur = 25;
boolean angenehm = temperatur >= 18 && temperatur <= 28;

// Boolean in Schleifen
boolean weitermachen = true;
while (weitermachen) {
    // Code
    weitermachen = false;  // Schleife beenden
}

// Vergleich mit Strings
String name = "Max";
boolean istMax = name.equals("Max");       // true
```

Worauf achten:

- Es gibt nur zwei Werte: `true` (wahr) oder `false` (falsch) - keine Zahlen!
- Alles klein schreiben: `true` und `false`, nicht `True` oder `False`
- In einem Array ist der Anfangswert immer `false`
- Zum Vergleichen: Bei Zahlen `==`, bei Text `.equals()`

char - Einzelnes Zeichen

Was ist char?

Der Datentyp `char` speichert genau ein einzelnes Zeichen (wie 'A', '5' oder '!').

Wertebereich:

- Alle Unicode-Zeichen (Buchstaben, Ziffern, Sonderzeichen)
- Von: 0 bis 65.535 (als Zahlenwert)
- Speichergröße: 16 Bit (2 Bytes)

Syntax:

```
char variablenName = 'Zeichen';  // EINFACHE Anfuehrungszeichen!
```

Beispiele:

```
// Deklaration und Initialisierung
char buchstabe = 'A';
char ziffer = '5';
char sonderzeichen = '!';
char leerzeichen = ' ';

// Mit Unicode-Werten
char euro = '\u20AC';                    //    -Symbol
char herzchen = '\u2764';                //

// Escape-Sequenzen
char neuzeile = '\n';                    // Zeilenumbruch
char tab = '\t';                         // Tabulator
char backslash = '\\';                   // \
char anfuehrung = '\'';                  // '

// Zeichen vergleichen
char note = 'A';
if (note == 'A') {
    System.out.println("Sehr gut!");
}

// Zeichen in Berechnungen (ASCII/Unicode-Werte)
char a = 'A';                            // Unicode-Wert: 65
char b = 'B';                            // Unicode-Wert: 66
int differenz = b - a;                   // differenz = 1
```

```java

// Kleinbuchstabe zu Gro buchstabe
char klein = 'a';                          // Unicode-Wert: 97
char gross = (char) (klein - 32);          // gross = 'A' (Wert: 65)

// Pr fen ob Buchstabe
char zeichen = 'M';
boolean istBuchstabe = (zeichen >= 'A' && zeichen <= 'Z') ||
                       (zeichen >= 'a' && zeichen <= 'z');

// Pr fen ob Ziffer
char z = '7';
boolean istZiffer = (z >= '0' && z <= '9');

// char aus String extrahieren
String text = "Hallo";
char erstes = text.charAt(0);              // erstes = 'H'
```

Worauf achten:

- Immer einfache Anführungszeichen verwenden: `'A'` ist richtig, Ä“ ist falsch
- Nur genau ein Zeichen erlaubt: `'A'` funktioniert, `'AB'` nicht
- `char` ist etwas anderes als `String` - ein char ist nur ein einzelnes Zeichen
- In einem Array ist der Anfangswert das NNull-Zeichen“(ein unsichtbares Zeichen)
- Jedes Zeichen hat intern eine Nummer - deshalb kannst du mit char auch rechnen

byte - Sehr kleine Ganzzahlen

Was ist byte?

Der Datentyp `byte` speichert sehr kleine ganze Zahlen. Wird selten benutzt, meist nur wenn man Speicherplatz sparen muss.

Wertebereich:

- Von: -128
- Bis: 127
- Speichergröße: 8 Bit (1 Byte)

Syntax:

```
byte variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
byte alter = 25;
byte temperatur = -15;
byte prozent = 100;

// H ufig bei Datei-Operationen
byte[] daten = new byte[1024];

// Konvertierung von int zu byte
int gross = 100;
byte klein = (byte) gross;

// Vorsicht bei  berschreitung
byte max = 127;
byte ueberlauf = (byte) (max + 1);         // ueberlauf = -128 (Overflow!)
```

Worauf achten:

- Sehr kleiner Wertebereich: nur -128 bis 127
- Wenn die Zahl zu groß wird, springt sie um (Überlauf)
- Wird meistens nur für spezielle Dinge verwendet
- In einem Array ist der Anfangswert immer `0`

short - Kleine Ganzzahlen

Was ist short?

Der Datentyp `short` speichert kleine ganze Zahlen. Ein Mittelweg zwischen `byte` und `int`.

Wertebereich:

- Von: -32.768
- Bis: 32.767
- Speichergröße: 16 Bit (2 Bytes)

Syntax:

```
short variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
short jahr = 2024;
short distanz = 5000;

// Konvertierung
int gross = 1000;
short klein = (short) gross;
```

Worauf achten:

- Wird selten verwendet, meist wird direkt `int` genommen
- Wertebereich: -32.768 bis 32.767
- Standard-Wert bei Arrays: `0`

long - Sehr große Ganzzahlen

Was ist long?

Der Datentyp `long` speichert sehr große ganze Zahlen - noch größer als int.

Wertebereich:

- Von: -9.223.372.036.854.775.808
- Bis: 9.223.372.036.854.775.807
- Speichergröße: 64 Bit (8 Bytes)

Syntax:

```
long variablenName = wert;
long grosseZahl = wertL;  // L am Ende fuer long-Literal
```

Beispiele:

```
// Deklaration und Initialisierung
long weltbevoelkerung = 8000000000L;       // L am Ende!
long millisekunden = System.currentTimeMillis();

// Ohne L wird es als int interpretiert (Fehler bei gro en Zahlen)
// long falsch = 8000000000;               // FEHLER!
long richtig = 8000000000L;                // Korrekt

// Berechnungen
long summe = 1000000000L + 2000000000L;
```

Worauf achten:

- Bei großen Zahlen musst du ein `L` ans Ende schreiben: `1000000000L`
- Ohne `L` denkt Java, es ist eine int-Zahl, und das kann schiefgehen
- In einem Array ist der Anfangswert immer `0L`
- Benutze long, wenn deine Zahlen größer als etwa 2 Milliarden werden können

float - Einfache Fließkommazahlen

Was ist float?

Der Datentyp `float` speichert Fließkommazahlen mit einfacher Genauigkeit. Weniger genau als `double`.

Wertebereich:

- Ca. 6-7 Dezimalstellen Genauigkeit
- Von: $\pm 1.4 \times 10^{-45}$
- Bis: $\pm 3.4 \times 10^{38}$
- Speichergröße: 32 Bit (4 Bytes)

Syntax:

```
float variablenName = wertF;  // F am Ende fuer float-Literal
```

Beispiele:

```
// Deklaration und Initialisierung
float pi = 3.14F;                          // F am Ende!
float temperatur = 36.5F;
float gewicht = 75.8F;

// Ohne F wird es als double interpretiert
// float falsch = 3.14;                    // Warnung/Fehler
float richtig = 3.14F;                     // Korrekt

// Berechnungen
float summe = 10.5F + 20.3F;
```

Worauf achten:

- `F` oder `f` am Ende anhängen: `3.14F`
- Weniger genau als `double`
- In der Praxis wird meist `double` bevorzugt
- Standard-Wert bei Arrays: `0.0F`

String - Zeichenketten

Was ist ein String?

Beschreibung

Ein `String` speichert Text - also mehrere Zeichen zusammen (wie "Halloöder MMax Mustermann"). String ist etwas Besonderes: Er gehört nicht zu den primitiven Datentypen.

Eigenschaften:

- Wenn du einen String einmal erstellt hast, kannst du ihn nicht mehr ändern - nur einen neuen erstellen
- Strings schreibst du mit doppelten Anführungszeichen: `TText"`
- Ein String kann so lang sein wie du willst

Syntax:

```
String variablenName = "Text";
```

Beispiele:

```
// Deklaration und Initialisierung
String name = "Max Mustermann";
String begruessung = "Hallo Welt!";
String leer = "";                          // Leerer String
String mitZahl = "Ich bin 25 Jahre alt";

// String-Konkatenation (Verkettung)
String vorname = "Max";
String nachname = "Mustermann";
String vollername = vorname + " " + nachname;  // "Max Mustermann"

// Mit Zahlen verketten
int alter = 25;
String text = "Ich bin " + alter + " Jahre alt";

// String-Methoden
String wort = "Hallo";
int laenge = wort.length();                // 5
char erstes = wort.charAt(0);              // 'H'
String gross = wort.toUpperCase();         // "HALLO"
String klein = wort.toLowerCase();         // "hallo"

```

```java
// Strings vergleichen
String a = "Hallo";
String b = "Hallo";
boolean gleich = a.equals(b);              // true
boolean gleichIgnoreCase = a.equalsIgnoreCase("HALLO");  // true

// WICHTIG: Nicht mit == vergleichen!
String s1 = "Test";
String s2 = new String("Test");
boolean falsch = (s1 == s2);               // false (vergleicht Referenzen)
boolean richtig = s1.equals(s2);           // true (vergleicht Inhalt)

// Teilstrings
String text2 = "Hallo Welt";
String teil = text2.substring(0, 5);       // "Hallo"
String ab = text2.substring(6);            // "Welt"

// Suchen in Strings
String satz = "Java ist toll";
boolean enthaelt = satz.contains("ist");   // true
int position = satz.indexOf("ist");        // 5
boolean beginnt = satz.startsWith("Java"); // true
boolean endet = satz.endsWith("toll");     // true

// Ersetzen
String original = "Hallo Welt";
String neu = original.replace("Welt", "Java");  // "Hallo Java"

// Leerzeichen entfernen
String mitLeer = "  Text  ";
String ohneLeer = mitLeer.trim();          // "Text"

// String in Array umwandeln
String liste = "Apfel,Birne,Orange";
String[] fruechte = liste.split(",");      // ["Apfel", "Birne", "Orange"]

// Zahlen zu String
int zahl = 42;
String zahlString = String.valueOf(zahl);  // "42"
// ODER
String zahlString2 = "" + zahl;            // "42"

// String zu Zahl
String nummerText = "123";
int nummer = Integer.parseInt(nummerText); // 123
double komma = Double.parseDouble("3.14"); // 3.14
```

Worauf achten:

- Doppelte Anführungszeichen verwenden: `TText“` ist richtig, `'Text'` ist falsch
- Zum Vergleichen von Strings immer `.equals()` benutzen, nicht `==`
- String schreibst du immer mit großem S am Anfang
- Wenn du einen String ändern willst, bekommst du eigentlich einen neuen String zurück
- `null` bedeutet “gar kein Text“, ist lleerer Text das ist ein Unterschied
- In einem Array ist der Anfangswert immer `null`

Typkonvertierung

Implizite Konvertierung (Automatisch)

Beschreibung

Java wandelt automatisch kleinere Zahlen in größere um. Zum Beispiel wird eine int-Zahl automatisch zu double, wenn nötig.

Beispiele:

```
// int zu double (automatisch)
int ganzzahl = 5;
double kommazahl = ganzzahl;                // kommazahl = 5.0

// byte zu int (automatisch)
byte klein = 100;
int gross = klein;                          // gross = 100

// char zu int (automatisch)
char buchstabe = 'A';
int code = buchstabe;                       // code = 65 (Unicode-Wert)

// In Berechnungen
int a = 5;
double b = 2.5;
double ergebnis = a + b;                    // ergebnis = 7.5
```

Explizite Konvertierung (Type Cast)

Beschreibung

Wenn du eine größere Zahl in eine kleinere umwandeln willst, musst du Java das ausdrücklich sagen. Dabei können Informationen verloren gehen!

Syntax:

```
zielTyp variable = (zielTyp) quellVariable;
```

Beispiele:

```
// double zu int (Nachkommastellen gehen verloren!)
double kommazahl = 3.7;
int ganzzahl = (int) kommazahl;             // ganzzahl = 3 (nicht 4!)

```

```java
// long zu int
long gross = 1000L;
int klein = (int) gross;

// int zu byte (Vorsicht bei gro en Zahlen!)
int zahl = 200;
byte kleinByte = (byte) zahl;              // Overflow!

// Division mit Cast
int a = 5;
int b = 2;
double ergebnis = (double) a / b;          // ergebnis = 2.5

// Runden statt Abschneiden
double wert = 3.7;
int gerundet = (int) Math.round(wert);     // gerundet = 4
```

Worauf achten:

- Wenn du double zu int umwandelst, werden die Nachkommastellen einfach weggeschnitten, nicht gerundet! (3.9 wird zu 3, nicht zu 4)
- Wenn die Zahl zu groß ist, können seltsame Werte herauskommen
- Zum Runden benutze `Math.round()`
- Automatisch: klein zu groß (z.B. int zu double)
- Mit Klammern: groß zu klein (z.B. double zu int)

Übersicht: Primitive Datentypen

Typ	Größe	Wertebereich	Standard
byte	8 Bit	-128 bis 127	0
short	16 Bit	-32.768 bis 32.767	0
int	32 Bit	-2.147.483.648 bis 2.147.483.647	0
long	64 Bit	ca. -9,2 $\times$ 10^{18} bis 9,2 $\times$ 10^{18}	0L
float	32 Bit	ca. $\pm$3,4 $\times$ 10^{38}	0.0F
double	64 Bit	ca. $\pm$1,7 $\times$ 10^{308}	0.0
char	16 Bit	0 bis 65.535 (Unicode)	'\u0000'
boolean	1 Bit	true oder false	false

Tabelle 1: Übersicht der primitiven Datentypen in Java

Arrays

Was ist ein Array?

Beschreibung

Ein Array ist wie eine Liste mit nummerierten Fächern. Jedes Fach kann einen Wert speichern. Wichtig: Alle Werte müssen vom gleichen Typ sein (z.B. nur Zahlen oder nur Text). Die Anzahl der Fächer ist fest.

Eigenschaften:

- Die Größe ist fest: Wenn du ein Array mit 5 Plätzen erstellst, bleiben es 5 Plätze
- Die Nummerierung beginnt bei 0: Das erste Fach hat die Nummer 0, das letzte die Nummer `length - 1`
- Alle Werte im Array müssen vom gleichen Typ sein
- Neue Arrays sind automatisch mit Startwerten gefüllt (0 für Zahlen, false für boolean, null für Text)

Syntax:

```
// Deklaration und Initialisierung mit  G r   e
datentyp[] arrayName = new datentyp[groesse];

// Deklaration und Initialisierung mit Werten
datentyp[] arrayName = {wert1, wert2, wert3};
```

Beispiele:

```
// Array mit fester G r   e erstellen
int[] zahlen = new int[5];        // 5 Pl tze, alle 0
String[] namen = new String[3];   // 3 Pl tze, alle null
boolean[] flags = new boolean[4]; // 4 Pl tze, alle false

// Array mit Werten erstellen
int[] noten = {1, 2, 3, 2, 1};
String[] wochentage = {"Mo", "Di", "Mi", "Do", "Fr"};
double[] preise = {9.99, 14.50, 7.25};

// Mehrdimensionale Arrays
int[][] matrix = new int[3][3];   // 3x3 Matrix
int[][] tabelle = {{1, 2}, {3, 4}, {5, 6}};
```

Worauf achten:

- Die Größe kannst du nach dem Erstellen nicht mehr ändern
- Die Nummerierung beginnt bei 0, nicht bei 1!
- Alle Werte müssen vom gleichen Typ sein
- Wenn du ein Array noch nicht erstellt hast, ist es `null`

array.length - Länge eines Arrays

Was ist array.length?

Mit `length` kannst du herausfinden, wie viele Plätze das Array hat. Wichtig: Bei Arrays schreibst du **keine** Klammern dahinter!

Eigenschaften:

- Ist ein Attribut, keine Methode: **ohne** Klammern **( )**
- Gibt die Größe zurück, mit der das Array erstellt wurde
- Funktioniert auch bei leeren Arrays (nur Standard-Werte)

Syntax:

```
int laenge = arrayName.length;  // OHNE Klammern!
```

Beispiele:

```
// L nge verschiedener Arrays
int[] zahlen = {10, 20, 30, 40, 50};
int anzahl = zahlen.length;                // anzahl = 5

String[] namen = new String[10];
int groesse = namen.length;                // groesse = 10 (auch wenn leer!)

// Leeres Array
int[] leer = new int[0];
int laenge = leer.length;                  // laenge = 0

// In Schleife verwenden
int[] werte = {1, 2, 3, 4};
for (int i = 0; i < werte.length; i++) {
    System.out.println(werte[i]);
}

// Letztes Element zugreifen
int[] liste = {100, 200, 300};
int letzter = liste[liste.length - 1];   // letzter = 300

// Bei mehrdimensionalen Arrays
int[][] matrix = new int[3][4];
int zeilen = matrix.length;                // zeilen = 3
int spalten = matrix[0].length;            // spalten = 4
```

Worauf achten:

- Bei Arrays: `.length` ohne Klammern
- Bei Strings: `.length()` mit Klammern - das ist ein häufiger Fehler!
- Gibt die Anzahl der Plätze zurück, nicht die Anzahl der gefüllten Plätze
- Auch wenn alle Werte noch 0 oder null sind, hat das Array trotzdem seine feste Länge

array[index] - Element lesen

Was ist array[index]?

Mit eckigen Klammern `[]` kannst du auf ein bestimmtes Fach im Array zugreifen. Die Zahl in den Klammern ist die Nummer des Fachs (beginnt bei 0).

Eigenschaften:

- Index beginnt bei 0: Erstes Element ist `array[0]`
- Letztes Element ist `array[array.length - 1]`
- Bei ungültigem Index: `ArrayIndexOutOfBoundsException`

Syntax:

```
datentyp element = arrayName[index];
```

Beispiele:

```
// Einzelne Elemente lesen
int[] zahlen = {10, 20, 30, 40, 50};
int erstes = zahlen[0];                    // erstes = 10
int zweites = zahlen[1];                   // zweites = 20
int letztes = zahlen[4];                   // letztes = 50

// Alle Elemente durchlaufen
String[] woerter = {"Hallo", "Welt", "Java"};
for (int i = 0; i < woerter.length; i++) {
    String wort = woerter[i];
    System.out.println(wort);
}

// Bestimmte Position pr fen
int[] noten = {1, 2, 3, 2, 1};
if (noten[2] == 3) {
    System.out.println("Dritte Note ist eine 3");
}

// Letztes Element
int[] liste = {5, 10, 15, 20};
int letzterWert = liste[liste.length - 1];  // letzterWert = 20

// Element in Berechnung verwenden
int[] preise = {100, 200, 150};
```

```
26 int summe = preise[0] + preise[1] + preise[2];  // summe = 450
27
28 // Mehrdimensionale Arrays
29 int[][] matrix = {{1, 2, 3}, {4, 5, 6}};
30 int wert = matrix[0][1];                  // wert = 2 (1. Zeile, 2. Spalte)
```

Worauf achten:

- Die Nummerierung beginnt bei 0, nicht bei 1!
- Die Nummer muss zwischen 0 und `array.length - 1` liegen
- Wenn die Nummer zu groß oder zu klein ist, stürzt das Programm ab
- Das letzte Fach erreichst du mit `array[array.length - 1]`

array[index] = wert - Element schreiben

Was ist array[index] = wert?

So kannst du einen Wert in ein bestimmtes Fach des Arrays legen (oder den alten Wert überschreiben).

Eigenschaften:

- Überschreibt den vorherigen Wert an dieser Position
- Der neue Wert muss zum Datentyp des Arrays passen
- Index muss gültig sein (0 bis `length - 1`)

Syntax:

```
arrayName[index] = neuerWert;
```

Beispiele:

```
// Array bef llen
int[] zahlen = new int[5];                // Array mit 5 Pl tzen, alle 0
zahlen[0] = 10;                           // Erstes Element auf 10 setzen
zahlen[1] = 20;                           // Zweites Element auf 20 setzen
zahlen[4] = 50;                           // Letztes Element auf 50 setzen
// Ergebnis: {10, 20, 0, 0, 50}

// Werte  ndern
String[] namen = {"Anna", "Bob", "Clara"};
namen[1] = "Max";                         // "Bob" wird zu "Max"
// Ergebnis: {"Anna", "Max", "Clara"}

// In Schleife bef llen
int[] quadrate = new int[5];
for (int i = 0; i < quadrate.length; i++) {
    quadrate[i] = i * i;
}
// Ergebnis: {0, 1, 4, 9, 16}

// Wert erh hen
int[] punkte = {10, 20, 30};
punkte[0] = punkte[0] + 5;                // Erstes Element um 5 erh hen
// ODER k rzer:
punkte[0] += 5;
// Ergebnis: {15, 20, 30}
```

```java

// Array "leeren" (auf Standard-Wert setzen)
int[] werte = {1, 2, 3, 4, 5};
for (int i = 0; i < werte.length; i++) {
    werte[i] = 0;
}
// Ergebnis: {0, 0, 0, 0, 0}

// Zwei Elemente tauschen
int[] liste = {10, 20, 30};
int temp = liste[0];                        // Zwischenspeichern
liste[0] = liste[2];                        // 30 nach vorne
liste[2] = temp;                            // 10 nach hinten
// Ergebnis: {30, 20, 10}

// Array aus Benutzereingabe f llen
int[] zahlenListe = new int[3];
for (int i = 0; i < zahlenListe.length; i++) {
    // zahlenListe[i] = scanner.nextInt();
}

// Mehrdimensionale Arrays
int[][] matrix = new int[2][2];
matrix[0][0] = 1;
matrix[0][1] = 2;
matrix[1][0] = 3;
matrix[1][1] = 4;
// Ergebnis: {{1, 2}, {3, 4}}
```

Worauf achten:

- Die Nummer muss zwischen 0 und `array.length - 1` liegen
- Wenn die Nummer ungültig ist, stürzt das Programm ab
- Der Wert muss zum Typ des Arrays passen (Zahl für int-Array, Text für String-Array usw.)
- Der alte Wert an dieser Stelle geht verloren - er wird komplett ersetzt

Häufige Fehler

String mit == vergleichen

```
// FALSCH
String a = "Hallo";
String b = "Hallo";
if (a == b) { }  // Vergleicht Referenzen, nicht Inhalt!

// RICHTIG
if (a.equals(b)) { }  // Vergleicht Inhalt
```

Integer-Division

```
// FALSCH - ergibt 2, nicht 2.5
int a = 5;
int b = 2;
double ergebnis = a / b;  // ergebnis = 2.0

// RICHTIG
double ergebnis = (double) a / b;  // ergebnis = 2.5
// ODER
double ergebnis = 5.0 / 2.0;  // ergebnis = 2.5
```

char vs. String verwechseln

```
// FALSCH
char buchstabe = "A";  // Fehler: doppelte Anfuehrungszeichen
String wort = 'Hallo';  // Fehler: einfache Anfuehrungszeichen

// RICHTIG
char buchstabe = 'A';   // einfache Anfuehrungszeichen
String wort = "Hallo";  // doppelte Anfuehrungszeichen
```

L und F vergessen

```
// FALSCH
long gross = 5000000000;  // Fehler: zu gross fuer int
float zahl = 3.14;        // Warnung: double wird zu float

// RICHTIG
long gross = 5000000000L;  // L am Ende
float zahl = 3.14F;        // F am Ende
```

Overflow nicht beachten

```
// Problem
byte b = 127;
b++;  // b = -128 (Overflow!)

int i = 2147483647;
i++;  // i = -2147483648 (Overflow!)
```

null vs. leerer String

```
String a = null;      // Keine Referenz (keine Methoden aufrufbar!)
String b = "";        // Leerer String (Methoden aufrufbar)

// a.length();        // FEHLER: NullPointerException
int laenge = b.length();  // Funktioniert: laenge = 0
```

String vs. Array - length

```
// FALSCH
String text = "Hallo";
int laenge = text.length;  // Fehler: Strings brauchen ()

int[] zahlen = {1, 2, 3};
int anzahl = zahlen.length();  // Fehler: Arrays haben KEINE ()

// RICHTIG
int laenge = text.length();    // String MIT Klammern
int anzahl = zahlen.length;    // Array OHNE Klammern
```

Array-Index-Grenzen

```
// FALSCH
int[] zahlen = {10, 20, 30};  // length = 3, Indizes: 0, 1, 2
int x = zahlen[3];            // Fehler! Index 3 existiert nicht
int y = zahlen[-1];           // Fehler! Negative Indizes nicht erlaubt

// RICHTIG
int letzter = zahlen[zahlen.length - 1];  // Index 2
int erster = zahlen[0];                   // Index 0
```

Array-Größe nach Erstellung ändern

```
// FALSCH
int[] zahlen = new int[5];
zahlen.length = 10;  // Fehler! length ist read-only

// RICHTIG - neues Array erstellen und Werte kopieren
int[] zahlen = new int[5];
int[] neuesArray = new int[10];
for (int i = 0; i < zahlen.length; i++) {
    neuesArray[i] = zahlen[i];
}
zahlen = neuesArray;  // Referenz auf neues Array
```